

Cloning the peer project

Create a fork of the Fabric repository using the GitHub Web interface. Next, clone your fork in the appropriate location, as follows:

```
cd $GOPATH/src
mkdir -p github.com/hyperledger
cd github.com/hyperledger
git clone https://github.com/<username>/fabric.git
```

Bootstrapping the VM using Vagrant

Now you're ready to launch Vagrant, by using the following command:

```
cd $GOPATH/src/github.com/hyperledger/fabric/devenv
vagrant up
```

Go get yourself some coffee... this will take a few minutes. Once complete, you should be able to ssh into the Vagrant VM just created.

```
vagrant ssh
```

Starting your containerised deployment of the Fabric network

First, clone the Fabric samples repository (*git clone https://github.com/hyperledger/fabric-samples*).

If you don't have Fabric installed in your environment, install it by using the following command:

```
cd fabric-samples/scripts
./fabric-preload.sh
```

Then proceed to start a basic Fabric network, as follows:

```
cd fabric-samples/basic-network
./start.sh
```

Where should the chaincode go?

A lot of developers mount the chaincode directly from the development folder — that's a terrible idea. What you should do is develop chaincode in your GOPATH under the vendor-name *github.com/yourproject/and-here-goes-your-chaincode-files*. You can also write a script, which will copy your chaincode under GOPATH and run a *go build* command to spit out the go-errors before running it in your Fabric tools.

Here's a simple script to copy your chaincode:

```
#!/bin/bash
echo "Copying your chaincode"
rm -rf $GOPATH/src/github.com/project-name/go & mkdir -p $GOPATH/src/github.com/project-name/go
cp -Rf chaincode/* $GOPATH/src/github.com/project-name/go/
```

If you want to build the code, just add the following command:

```
#!/bin/bash
cd $GOPATH/src/github.com/project-name/go/
echo "Building chaincode"
go build.
```

Installing the chaincode

Now it's time to launch our Armageddon. We have our network running, so let's install the chaincode using the bash script we wrote, as follows:

```
#!/bin/bash
# install-chaincode.sh

LANGUAGE=golang
CC_SRC_PATH=github.com/project-name/go
CC_VERSION=1.0
docker exec -e "CORE_PEER_LOCALMSPID=Org1MSP" -e "CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp" cli peer chaincode install -n project-name -v $CC_VERSION -p "$CC_SRC_PATH" -l "$LANGUAGE"

docker exec -e "CORE_PEER_LOCALMSPID=Org1MSP" -e "CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp" cli peer chaincode instantiate -o orderer.example.com:7050 -C mychannel -n project-name -l "$LANGUAGE" -v $CC_VERSION -c '{"Args":["Org1MSP.member"]}'
```

That's all! In case you are wondering why a bash script has to be written for every small process, you can also write a single bash master script to control everything with logical conditions.

Updating the chaincode

This is where developers waste time — by updating the code, building it and deploying it again by updating the chaincode-version. Doing it in our style, let's write a script. I have the script added to one of my personal repos and the link is provided in *References*.

That's it. Run your queries and see the magic of your blockchain network with Hyperledger Fabric. **END** 

References

- [1] <https://github.com/hyperledger/fabric>
- [2] <https://bit.ly/2SWgLM6>

By: Swapnil Kulkarni

The author is experienced in blockchain, cloud native solutions, containers and enterprise software product architectures. He has vast experience with different cloud architectures. He is a technology hobbyist and writes at cloudnativetech.wordpress.com.